Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S

Clean coding

Coding principles

- DRY: don’t repeat yourself
- OneTask: do only one thing in a function/class
- Meaningful names
- No littering
- No magic constants
- const correctness

DRY – don’t repeat yourself

```
void doThis() {
  X -
  -
  -
  Y -
  -
  -
  Z -
  -
  -
}
```

```
void doThat() {
  X -
  -
  -
  Y -
  -
  -
}
```

```
void doThis2() {
  doX();
  doY();
  doZ();
}
```

```
void doThat2() {
  doX();
  doY();
}
```

OneTask: do one thing in a function

```
void doThis(a, b, c, d, e) {
  X -
  -
  -
  Y -
  -
  -
  Z -
  -
  -
}

void doThis2(a, b, c) {
  doX(a, b);
  doY(b);
  doZ(c);
}
```

Meaningful names

Code is written for humans first, computers second

Functions: verbNoun

Variables: two or more nouns

- include unit of measure (liters/secs/kg)
- Long scope => long name
  - For (i ...) only in loop scope => short name
  - for longer scopes use more descriptive name
- Limit the lifetime (scope) of variables
- Do not use Hungarian notation szFilename, ui32_counter
  - Use strong types instead
- Use consistent names (begin/end, isDone, ...)

No littering

Code is always up to date

Comments are not

- Try and put more descriptions into code
  - Fewer into comments
- Remove commented out code
  - Use the versioning system (git) instead
- Use comments to explain why,
  let the code explain how/what

No magic constants

Magic constants are naked numbers/strings in your code

```
a += 100; //why 100, not 101? what is 100?
if ('zorg' == name) { ... //hidden backdoor?
```

Name constants instead

- `const int LevelOffset = 100; //Levels are 100 units apart`
- `const auto superuser = 'zorg'; //definitely a backdoor`

- Makes the intention clear
- Constants defined in a single place (DRY)
  - Easy to update
- You can document named constants

Const correctness

Java: @ReadOnly, final

C++/Rust: const

Python: not supported

Allows for pure functions

- No side effects

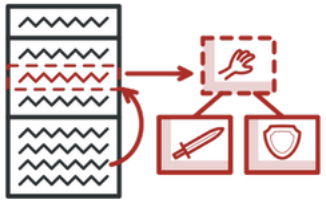
const/final variables

- Easy to reason about
  - will keep initial value
  - simplifies debugging
  - compiler optimization will fold extra variables

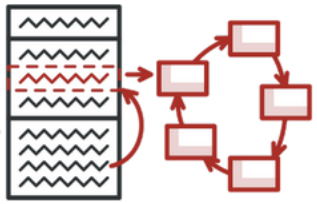
Behavioral patterns



Command
Action objects



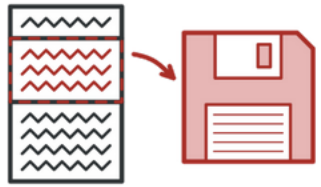
Strategy
Object per algorithm



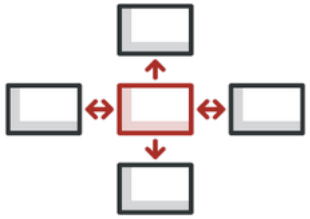
State
Finite state machine



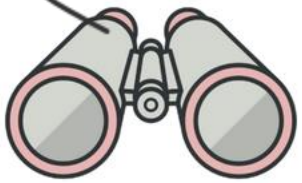
Template
Algorithm blueprint



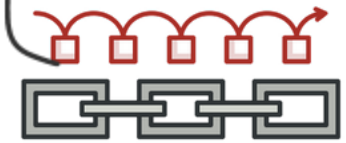
Memento
Snapshot/undo



Mediator
Reduce cross-talk



Observer
Subscribe to events



Chain of responsibility
Message passing



Visitor
Command++



Iterator
Visit every element

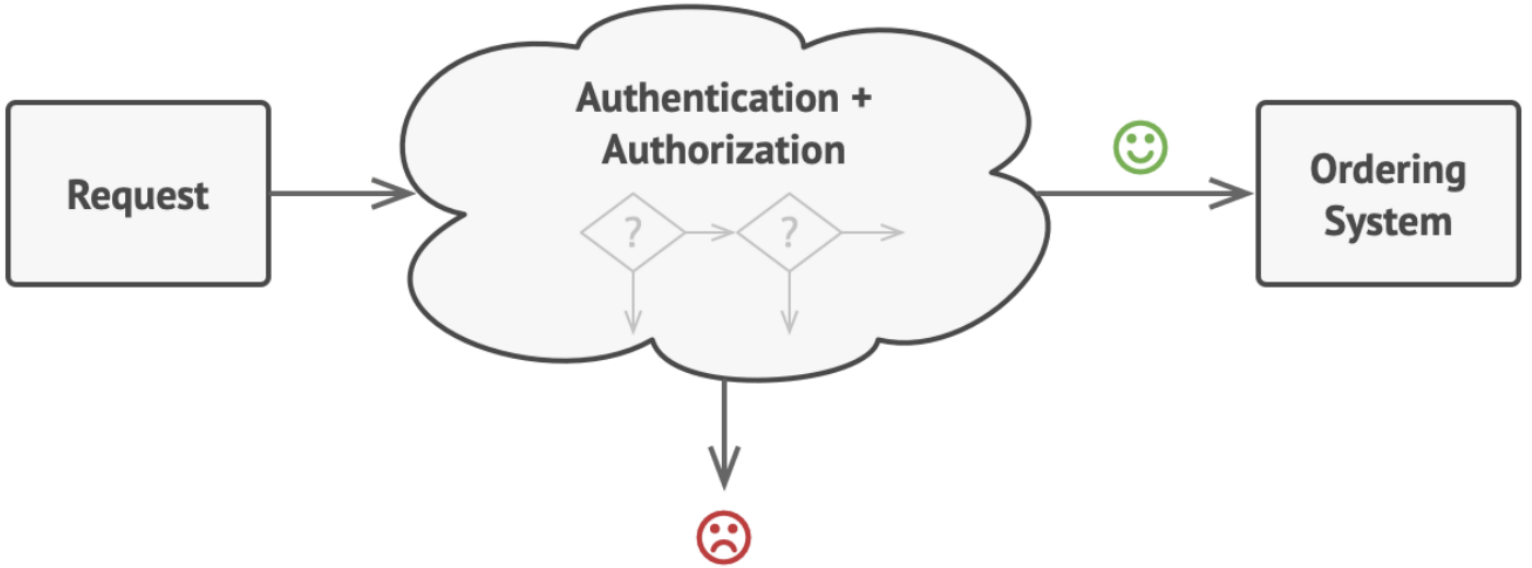
Chain of responsibility

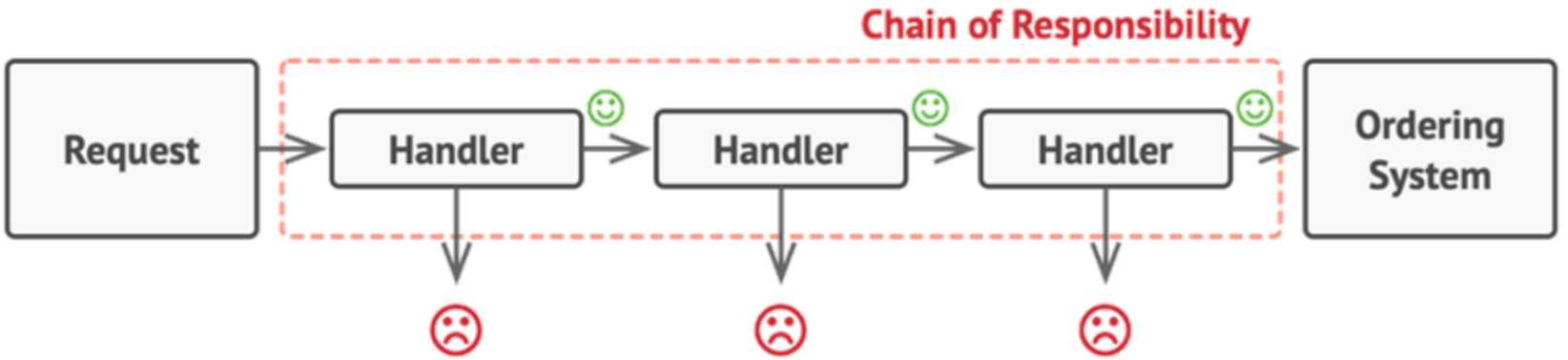


Receive a request

Process it

Pass it on, or drop it



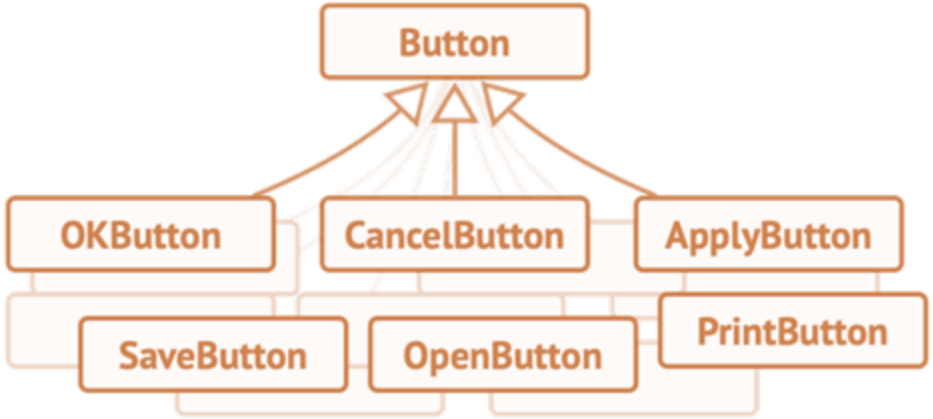


Command

Separate commands from UI

OKButton IS_A Button? => Yikes

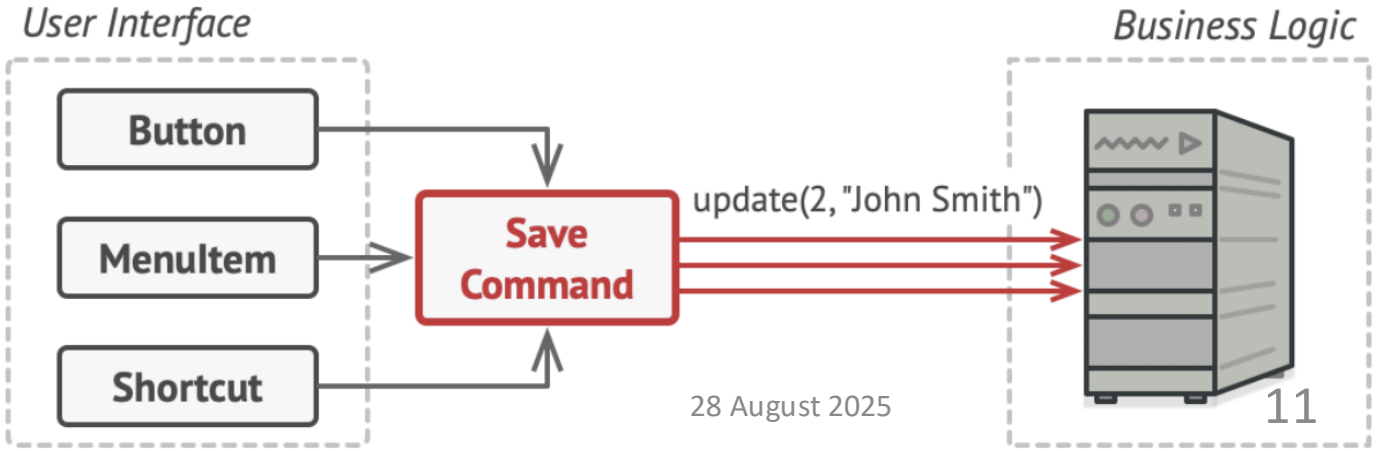
- Confusion of property/class
- Explosion of classes
- Business logic everywhere



- Duplicated code
- Tight coupling UI - logic



- No code duplication
- Loose coupling



Command

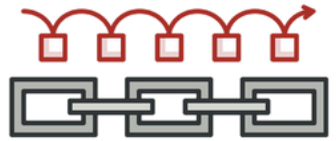


```
type Command: []{}-> bool; //lambda returning bool
const emptyCommand: Command = [&]{ return false; };

class UIControl: public InvisibleControl {
public
  action: Command = emptyCommand; //do nothing;
  bool execute() final {
    return action();
  }
}
//                                +--- [&] = capture context
form1.Actions.Add(ID_SaveAs, [&]{
  if (savedialog1.execute()) { //true = user pressed OK on SaveDialog
    auto filename = savedialog1.filename;
    editor1.SaveToFile(filename);
  });
```

Every modern language has lambdas, no longer needs OOP workaround

Chain of responsibility: WinAPI messages



Receive a request

Process it

Pass it on, or drop it

Form1
└ Panel1
  └ Button1



Option 1: every link in chain knows next link

```
Msg 1: Button1.OnMouseOver => draw glow effect => drop msg
Msg 2: Panel1.OnMouseOver => do nothing => call owner.OnMouseOver
          Form1.OnMouseOver => Flash caption => no owner => drop msg
```

Chain of responsibility: WinAPI messages

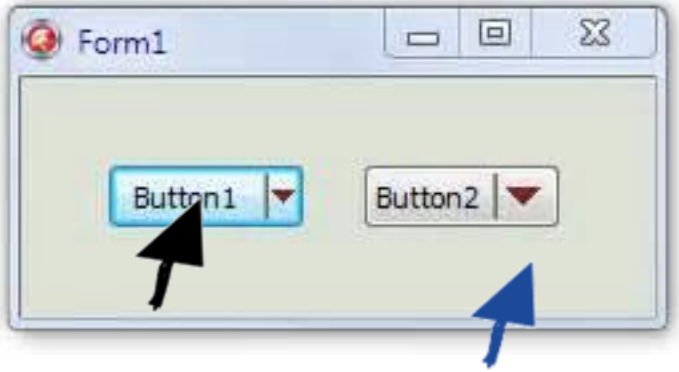


Receive a request

Process it

Pass it on, or drop it

Form1
└ Panel1
  └ Button1



Option 2: links do not know each other

WinAPI iterates (iterator pattern) over controls topmost -> bottom most

Traverse tree: do hit test, if hit,
then traverse child until no more hit = topmost control

Traverse hit controls in reverse order

```
Msg 1: Button1.OnMouseOver => draw glow effect => return false;
Msg 2: Panel1.OnMouseOver => do nothing => return true
       Form1.OnMouseOver => Flash caption => return false;
```

Iterator



Walk through every item in a collection

Collection: first + next + current methods

Iterator: `for (a in Collection) { a.dostuff() }`

=> for in loop runs the following pseudocode

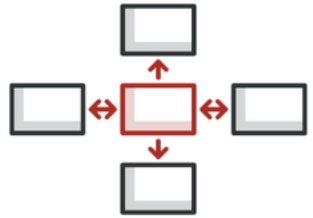
```
c.First();                  //current = c[-1]
while (c.next()) {  //current = c[0,1,2,...]
   a = c.current(); a.dostuff();
}
```

Design custom collections with `first, next, current` methods

Every modern language has for in loops.
No longer needs an OOP workaround

Mediator

Fancy version of command pattern



Memento

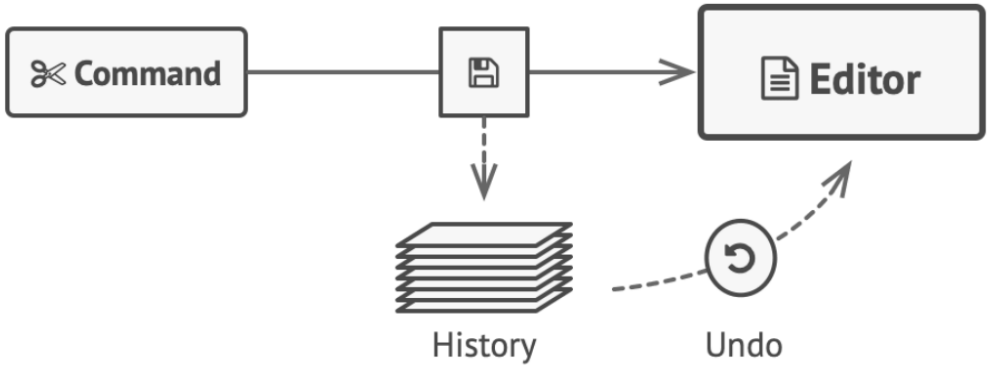


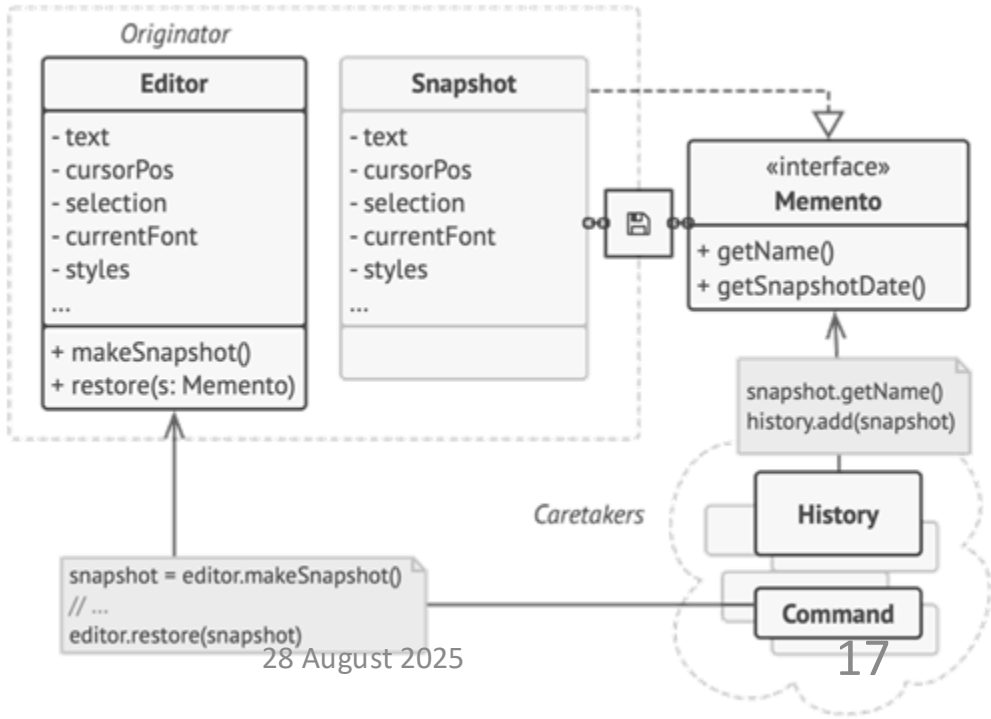
Program needs to save state from ≥ 1 objects

- Must not access object's private details
- Cloning objects not an option
  - Unsafe due to links with other objects

Solution

- Object creates '`snapshot`' to store state
- `Memento` adds metadata (date, command)
- `Caretaker` stores snapshots





Memento



```
class Editor implements IUndo {
  String text;   //gets saved
  Window owner;  //not saved
  EditorSnapshot makeSnapshot(); //IUndo
  void restoreSnapshot(EditorSnapshot restorePoint); //IUndo
}

class EditorSnapshot implements IMemento immutable {
  String text;
}

interface IMemento immutable {
  property int id;
  property IUndo source;
}

interface IMementoList immutable {  //if multiple objects must be restored
  property int id;
  property Datetime timestamp;
  property CommandId reason;
  property List<IMemento> sources;
  void restore() { for (s in sources) { s.source.restoreSnapshot(s); }
}

class Caretaker { //manages snapshots
  List<IMementoList> snapshots;
  void add(IMementoList item);
  void restore(int id) {
    restorepoint = getById(snapshots, id);
    restorepoint.restore();
```

Memento



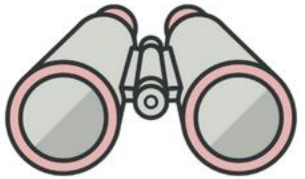
Useful for

- Undo (revert back to previous savepoint)
- Transactions (commit/rollback)
- Save to file

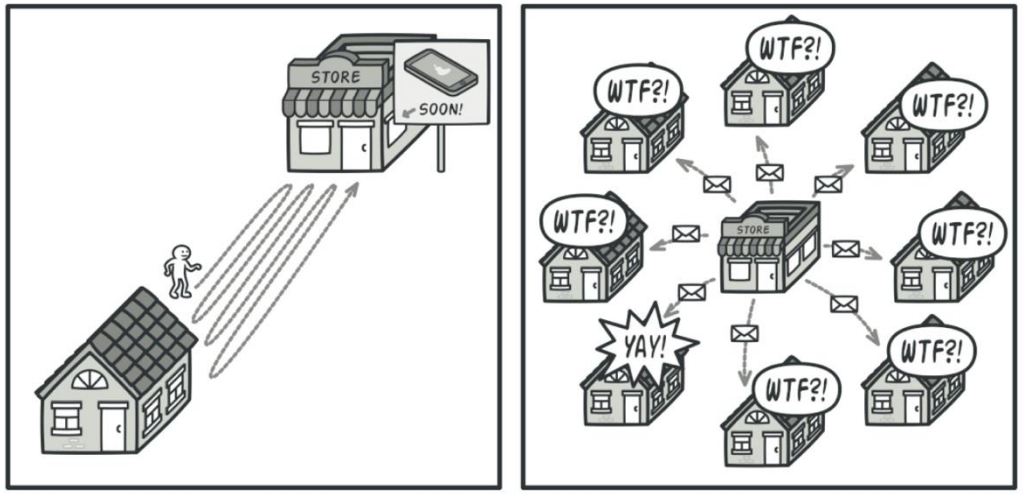
Pro	Con
Can save state, without having to know internal details of objects	Can consume a lot of RAM if too many snapshots
Caretaker tracks snapshots	May need to track original object, if these are not permanent, to destroy obsolete snapshots
	Needs immutable objects

Observer

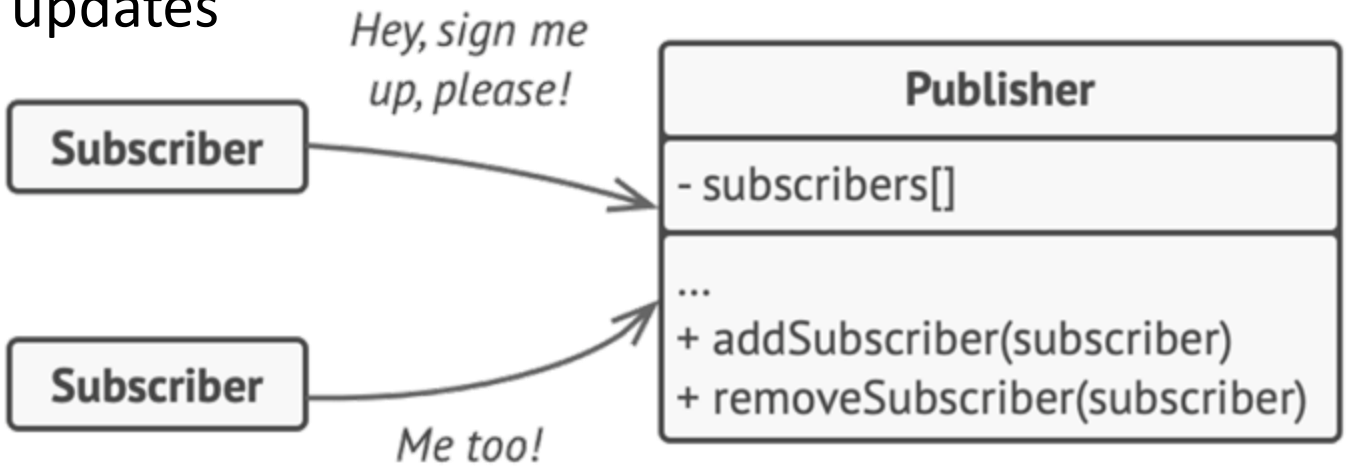
Update subscribers on the state of an object



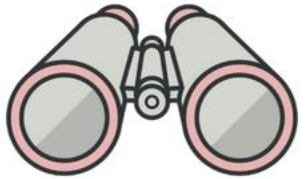
Problem: waiting for iphone 35 to arrive
Client could check in every day with the store
Store can email all possible clients on arrival



Solution: interested objects subscribe to updates



Observer



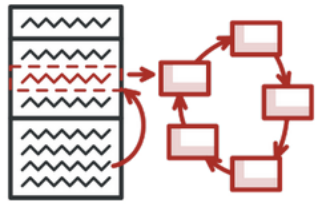
```
interface IObserver {
  void update(msg);
}

interface IPublisher {
  void subscribe(IObserver client);
  void unsubscribe(IObserver client);
}

class Publisher implements IPublisher {
  List<IObserver> clients;
  private void update(msg) {
    for (c in clients) { c.update(msg); }
  }
```

State

Finite state machine



Problem:

- complex case statement controls state

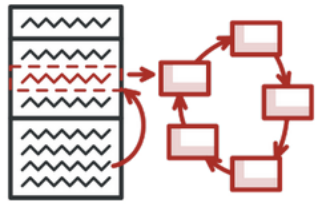
```
class Document is
    field state: string
    // ...
    method publish() is
        switch (state)
            "draft":
                state = "moderation"
                break
            "moderation":
                if (currentUser.role == "admin")
                    state = "published"
                break
            "published":
                // Do nothing.
                break
    // ...
```

Solution:

- IState interface
- One class per state
- Master object switches state
  - Calls state object to do state work

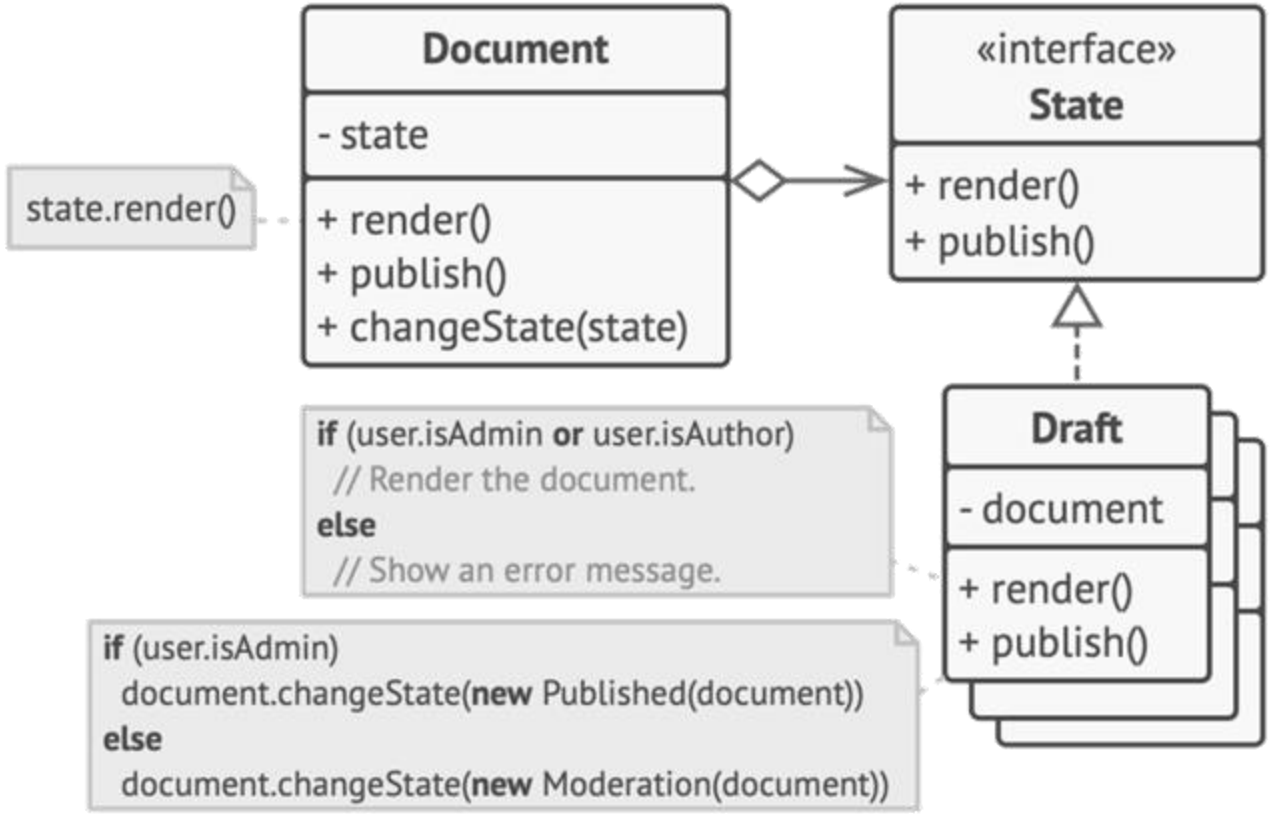
State

Finite state machine

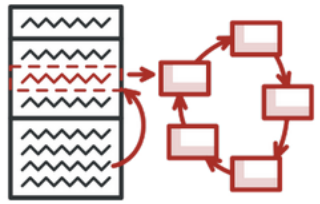


Solution:

- IState interface
- One class per state
- Master object switches state
  - Calls state object to do state work



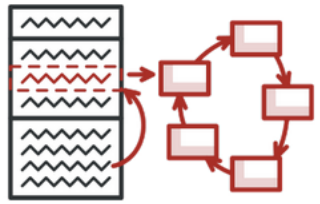
State: music player



```
interface IPlayState {
  property AudioPlayer player;
  void pressPlay(); //play/pause
}
class PlayState implements IPlayState {
  void pressPlay() { //from play -> pause
    player.setState(new PauseState());
    player.pauseMusic();
  }
}
class PauseState implements IPlayState {
  void pressPlay() {//from pause -> play
    player.setState(new PlayState()); }
    player.playMusic();
  }
}
class MusicPlayer {
  IPlayState state = new PauseState();
  Button playButton(state.pressPlay);
}
```

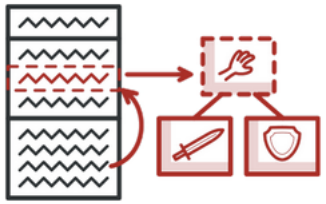
State: music player

States with shared code can inherit from or contain shared classes.



Pro	Con
Single responsibility: One state per class	Overkill if only few simple states
Open/closed: can change state details without having to change player class	
Eliminate complex state machine details (Master object can only be in a single state)	

Strategy



More complex version of State

- out of scope -

Template

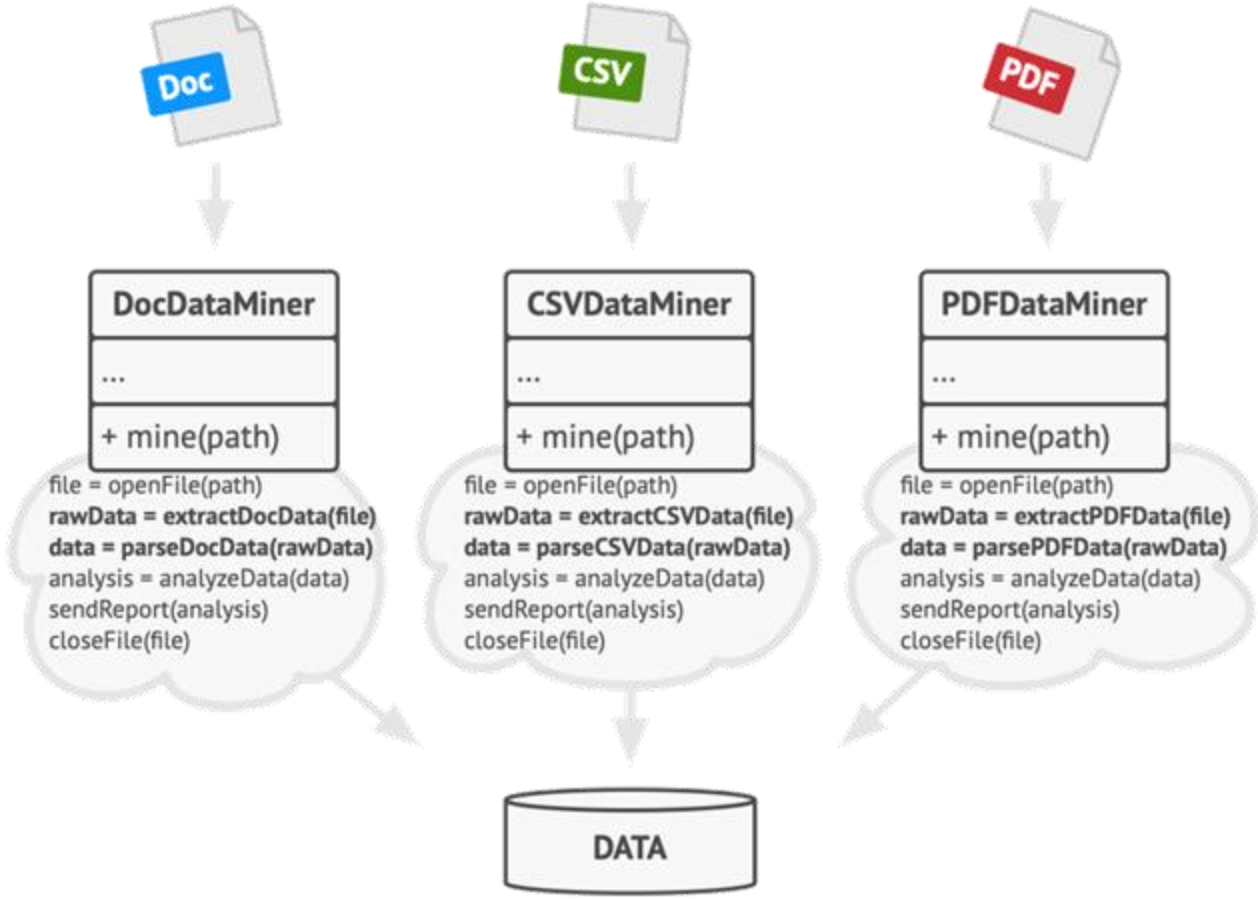


Define the skeleton of an algorithm

But not the detailed steps

Problem:

- Different versions of algorithm
- Repeated code in each version
  - Violates DRY principle



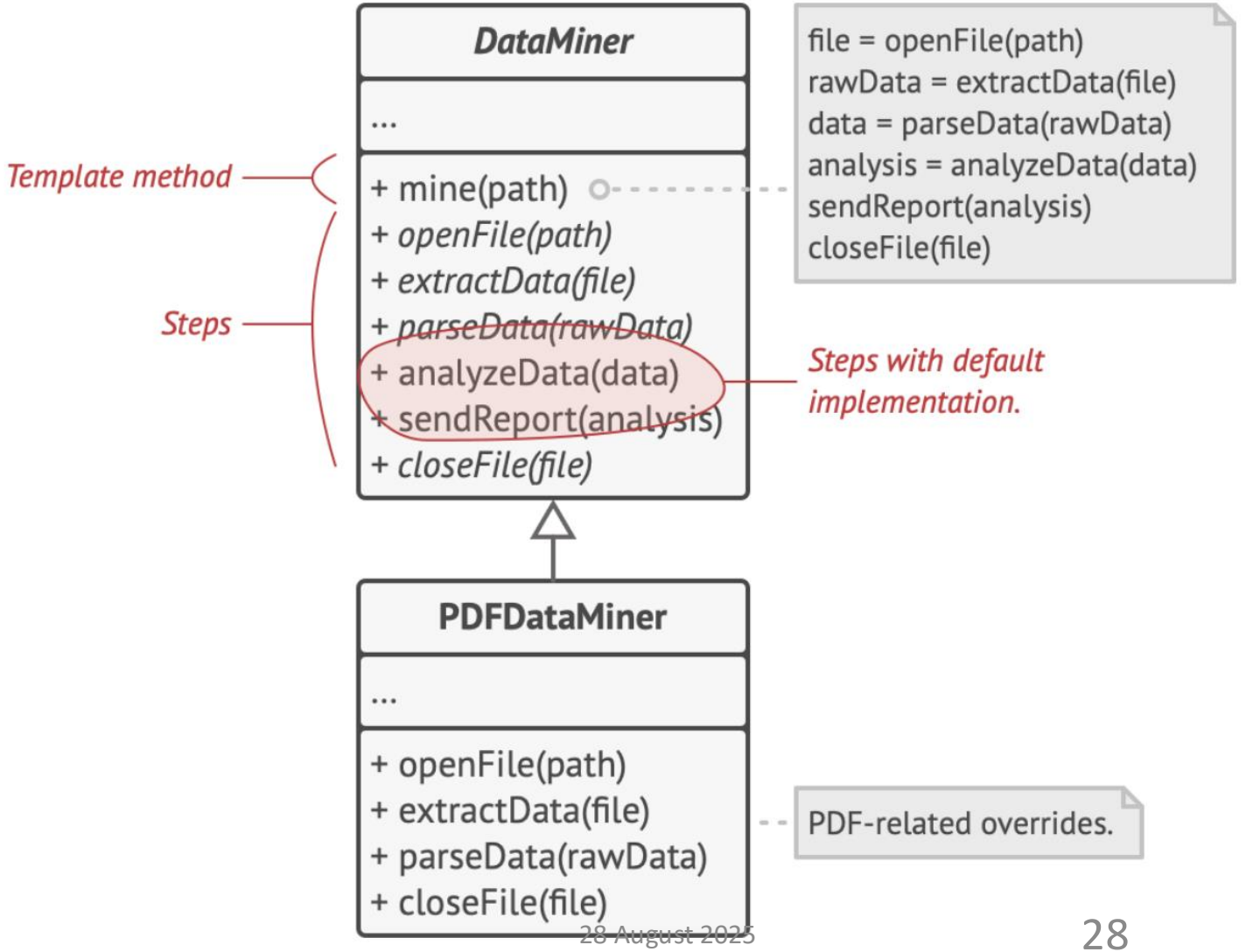
Template

Define the skeleton of an algorithm

But not the detailed steps

Solution:

- Put steps in class A
- Put details in class B



Template



```
class BrewMaster {
  void boilWater() { print("boil water"); }
  void addPlants() { /*do nothing*/ }
  void removePlants() { /*do nothing*/ }
  void addCondiments() { /*do nothing*/ }
}
class BrewTea extends BrewMaster {
  void addPlants() { hang tea bag }
  void removePlants() { remove tea bag }
  void addCondiments() { just sugar, no milk }
}
class BrewCoffee extends BrewMaster {
  void addPlants() { add coffee }
  //no sugar, no milk (duh)
}
```

Template



Define the skeleton of an algorithm

But not the detailed steps

Solution:

- Put steps in class A
- Put details in class B

Pro	Con
You can let clients override only certain parts of a large algorithm, making them less affected by changes that happen to other parts of the algorithm.	Overkill if only few simple states
Open/closed: can change state details without having to change player class	
Eliminate complex state machine details (Master object can only be in a single state)	

Visitor



More complex version of command

See lecture ASD15.

Examples

- Builder pattern: https://godbolt.org/z/Md6GWrPxh